

Comparação dos Resultados

Métricas de código antes e depois da refatoração de code smells

`too-many-statements`

`too-many-branches`

`too-many-instance-attributes`

Graziella Rodrigues e Wania Santos

O que é o Piccolo?

Visão geral e propósito simplificado do ecossistema utilizado na atividade de manutenção de software.

Definição

ORM & Query Builder

É um projeto **open source** desenvolvido em Python que oferece um ORM (Object-Relational Mapper) e um query builder moderno, rápido e amigável ao desenvolvedor.

Foco

Performance & Flexibilidade

Garante o desenvolvimento focado em:

- **Alta performance** de execução.
- Suporte completo a operações **síncronas** e **assíncronas** nativas.

Objetivo

Produtividade Clara

Combinar produtividade com **clareza de código**, oferecendo uma alternativa moderna, robusta e tipada frente aos ORMs tradicionais do ecossistema.

Objetivo da comparação

Avaliar o impacto das refatorações na qualidade interna do código.



Pergunta central

A refatoração reduziu os code smells sem prejudicar a manutenção, os testes e as demais métricas?

Métricas consideradas

Radon: medir complexidade e manutenibilidade

Pylint: identificar smells e score do projeto

CodeCarbon: medir impacto ambiental

Pytest: testes que passaram e falharam

Antes da refatoração

O projeto tinha testes estáveis, mas trechos com alta complexidade e smells estruturais.

40

maior complexidade
rank E

30

smells-alvo
 $6 + 13 + 11$

0

testes falhando
711 passaram

7,91

score Pylint
/10

Indicadores Críticos

Função mais complexa `create_pydantic_model` — CC 40

Pylint refactor 318 ocorrências

Diagnóstico Geral

Leitura da Situação

Havia uma boa base de testes, mas a estrutura interna indicava pontos de difícil manutenção.

O projeto possuía estabilidade técnica garantida pelos testes, mas apresentava altos riscos de manutenibilidade futura devido à complexidade acumulada.

Processo de refatoração

As mudanças se concentraram em dividir responsabilidades e reduzir excesso de lógica em um único bloco.

1

Identificar

Selecionar ocorrências do Pylint ligadas a estrutura.

2

Extraír métodos

Separar blocos grandes em funções auxiliares coesas.

3

Agrupar atributos

Encapsular dados relacionados para reduzir classes inchadas.

4

Reexecutar métricas

Comparar antes e depois com Radon, Pylint, Pytest e CodeCarbon.

Estratégia de Execução

Objetivo de Redução

As mudanças se concentraram em dividir responsabilidades e reduzir excesso de lógica em um único bloco, atacando diretamente a complexidade.

Medição de Impacto

A tabela ao lado ilustra a volumetria inicial antes de iniciarmos as rodadas sucessivas de refatoração de código.

Resultado principal: smells-alvo reduziram

A refatoração foi efetiva no foco escolhido para a atividade.

-100%

too-many-statements

6 → 0 (eliminado)

-92%

too-many-branches

13 → 1 (quase eliminado)

-73%

too-many-instance-attributes

11 → 3 (redução significativa)

Métricas de Smells			
Smell	Antes	Depois	Impacto
too-many-statements	6	0	eliminado
too-many-branches	13	1	quase eliminado
too-many-instance-attributes	11	3	redução signif.

Síntese & Impacto

Redução Crítica de Ocorrências

Os problemas estruturais diretamente atacados diminuíram drasticamente de 30 para apenas 4 ocorrências no total.

Eficácia do Foco Escolhido

A estratégia de refatoração concentrada provou-se altamente eficaz, limpando quase por completo os principais gargalos de manutenção do projeto.

Complexidade e manutenibilidade

Os piores casos de complexidade melhoraram, mas o índice de manutenibilidade caiu levemente.

40 → 19

maior complexidade
melhora expressiva

7 → 0

ranks D/E
eliminados

82,51 → 82,15

MI médio
leve queda

Métricas por Função

Função / método	Antes	Depois
create_pydantic_model	40	7
_run_alter_columns	29	4
get_snapshot	25	3
Select.default_querystrings	19	7

Análise & Interpretação

Redistribuição de Lógica

A lógica foi redistribuída em unidades menores, reduzindo a dificuldade dos trechos mais críticos.

Impacto no MI (Manutenibilidade)

A queda observada no MI é explicada pelo aumento de tamanho em alguns arquivos auxiliares criados.

Trade-offs observados

A melhoria estrutural veio acompanhada de crescimento no código e de Pylint global levemente pior.

+4,09%

crescimento de LOC
mais helpers

-7

categoria refactor
melhorou

-0,02

score Pylint
leve queda

Métricas brutas

LOC:25.642 → 26.692
LLOC:10.889 → 11.461
SLOC:15.993 → 16.904
Comentários:879 → 797

Pylint

Score:7,91 → 7,89

Análise & Impacto

Leitura Principal

O objetivo local foi atingido, mas ainda há avisos secundários para limpar.

Trade-offs Identificados

Crescimento de Código: A melhoria estrutural trouxe um crescimento do código (+4,09% de LOC).

Estabilidade Pylint: Observou-se uma leve queda global no score e nos avisos secundários.

Pytest e CodeCarbon

A validação funcional teve pequena regressão; as métricas ambientais melhoraram na medição.

711 → 708

testes passaram
leve queda

0 → 3

testes falharam
investigar

-10,1%

emissões
CodeCarbon

Métrica	Antes	Depois
Duração	2,872s	2,851s
Emissões	1,42e-06	1,28e-06
Energia	1,45e-05	1,30e-05

Conclusão

A refatoração foi positiva, mas não completamente estabilizada.

Melhorou

- Smells-alvo reduziram fortemente.
- Complexidade máxima caiu de 40 para 19.
- Funções rank D/E foram eliminadas.

Exige atenção

- Código ficou maior.
- Score Pylint caiu levemente.
- 3 testes passaram a falhar.

Leitura final

A qualidade estrutural **melhorou nos pontos refatorados**, principalmente pela redução de métodos extensos e complexos.

Ainda assim, a refatoração gerou **efeitos colaterais**: aumento de tamanho, leve queda no Pylint e falhas no Pytest.